Gymnázium, Praha 6, Arabská 14
Programování

Ročníkový projekt

Pong pro dva hráče v Javě

Jakub Tůma, 1E

Duben 2026

Prohlašuji, že jsem jediným autorem tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000 Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

V Praze dne .....

Anotace

Tato ročníková práce popisuje návrh a implementaci jednoduché počítačové hry typu Pong v jazyce Java s využitím knihovny JavaFX. Program umožňuje hru dvou hráčů na jedné klávesnici, zobrazuje aktuální skóre a po dosažení vítězného počtu bodů vyhlásí vítěze. Klíčovou částí řešení je herní smyčka s pravidelnou aktualizací stavu, detekce kolizí míčku s pálkami, okraji hrací plochy a výpočet odrazu míčku podle místa zásahu na pálce (změna úhlu i rychlosti).

Abstract

This year's project describes the design and implementation of a simple computer game Pong in the programming language of Java and the library JavaFX. The program allows the game of two players on one keyboard, it visualizes the current score and after reaching the winning point it announces the winner. The key component of the solution is a game loop with regular state updates, detection of the ball collision with the paddles, edges of the play area and the calculation of the ball bounce depending on the place of the collision with the paddle (change of the angle and speed).

Zadání projektu

Pong pro dva hráče v Javě je klasická arkádová hra, ve které každý hráč ovládá svou vlastní pálku – jeden na levé a druhý na pravé straně obrazovky. Hráči používají různé klávesy (např. W/S pro levého hráče a ↑/↓ pro pravého) a snaží se odrážet míček zpět k soupeři. Míček se neustále pohybuje po herním poli, odráží se od horní a dolní stěny a při kontaktu s pálkou mění směr. Pokud jeden z hráčů míček mine, soupeř získává bod.

Obsah

1 Úvod 6
2 Popis problematiky a cíle projektu 6
3 Použité technologie 6
4 Návrh řešení a architektura programu 7
4.1 Přehled tříd 7
4.2 Vztahy mezi třídami 7
5 Algoritmy a výpočty 8
5.1 Herní smyčka 8
5.2 Odraz míčku od horního a dolního okraje 8
5.3 Odraz míčku od pálek 9
5.4 Skórování a reset míčku 10
6 Uživatelské rozhraní a ovládání 10
6.1 Ovládání 10
6.2 Zobrazení 11
7 Požadavky a spuštění 11
7.1 Systémové požadavky 11
7.2 Instalace 12
7.3 Spuštění 12
8 Závěr 13
9 Citace 14

1 Úvod

V této práci se zabývám vytvořením jednoduché 2D počítačové hry Pong. Cílem práce je naprogramovat funkční aplikaci pro dva hráče, která zahrnuje plynulý pohyb objektů, detekci kolizí a systém skórování. V následujících kapitolách jsou popsány použité technologie, návrh tříd, klíčové algoritmy a způsob spuštění a ovládání aplikace.

2 Popis problematiky a cíle projektu

Pong je jedna z historicky prvních počítačových her vůbec. Poprvé byla vydána společností Atari v roce 1972 a stala se symbolem zrodu herního průmyslu. Přestože jde o graficky velmi jednoduchou hru, skrývá v sobě celou řadu zajímavých technických problémů, které jsou dodnes základem moderního herního programování.

Princip hry je přímočarý: dva hráči ovládají pálky na levé a pravé straně obrazovky a snaží se odrazit míček tak, aby protihráč nestačil zareagovat. Pokud hráč míček nezasáhne a ten projde za jeho pálku, soupeř získá bod. Hra končí, jakmile některý z hráčů dosáhne stanoveného počtu bodů.

I přes zdánlivou jednoduchost klade implementace hry tohoto typu na programátora konkrétní technické nároky. Základním problémem je zajistit plynulý a pravidelný pohyb všech herních objektů bez ohledu na výkon počítače – k tomu slouží herní smyčka. Dalším klíčovým problémem je detekce kolizí: program musí správně vyhodnotit, kdy a kde se míček dotkl pálky nebo okraje hracího pole, a odpovídajícím způsobem změnit jeho směr. Aby hra nebyla mechanicky předvídatelná, záleží na výsledném úhlu odrazu na přesném místě zásahu na pálce.

Cílem tohoto projektu je implementovat plně funkční verzi pro dva hráče hry Pong s důrazem na správnou fyziku pohybu míčku, přirozeně působící odraz od pálek a přehledné uživatelské rozhraní.

3 Použité technologie

- **Programovací jazyk:** Java
- **GUI / grafika:** JavaFX **[2]**
- **Běhové prostředí:** JVM (Java Runtime)
- **Vykreslování:** `Canvas`, `GraphicsContext`
- **Animace / herní smyčka:** `AnimationTimer` **[3]** (volá metodu handle() opakovaně v čase)
- **IDE:** IntelliJ IDEA 2025.3.2 **[4]**
- **JDK:** OpenJDK 25 (Oracle OpenJDK 25.0.2), language level 25 **[5]**

4 Návrh řešení a architektura programu

Program je rozdělen do několika tříd v balíčku `com.example.pong`.

4.1 Přehled tříd

- `Launcher` – Obsahuje metodu main, která spustí JavaFX aplikaci zavoláním `Application.launch(GameApplication.class, args)`.
- `GameApplication` - V metodě start(Stage stage) vytvoří okno, Canvas (800×600), nastaví obsluhu kláves a spustí herní smyčku pomocí AnimationTimer. V každém snímku volá:
  - gamePanel.update() – logika hry,
  - gamePanel.render(gc) – vykreslení.
- `GamePanel` – Obsahuje kompletní herní stav: pálky, míček, skóre, logiku kolizí, restart a vykreslování UI prvků (pozadí, skóre, text vítěze).
- `Paddle` – Datová třída pálky: pozice x, y, rozměry width, height a metoda draw(gc).
- `Ball` – Datová třída míčku: pozice x, y, rychlosti speedX, speedY, velikost size a metoda draw(gc).

4.2 Vztahy mezi třídami

- `GameApplication` **vlastní** instanci `GamePanel` (kompozice).
- `GamePanel` **vlastní** dvě instance `Paddle` a jednu instanci `Ball`.
- `Paddle`, `Ball` neřeší logiku hry – pouze drží stav a umí se vykreslit.

5 Algoritmy a výpočty

Nejdůležitější částí projektu je výpočet pohybu a kolizí míčku.

5.1 Herní smyčka

Každý snímek probíhá:

1) **Update:** pohyb pálek a míčku, vyhodnocení kolizí, skóre.
2) **Render:** vykreslení pozadí, skóre, pálek, míčku a textů.

5.2 Odraz míčku od horního a dolního okraje

Tento úsek kódu zajišťuje odraz míčku od horního a dolního okraje herního okna.

Program kontroluje, zda míček nepřekročil povolenou oblast na ose Y. Pokud ano, dojde ke změně směru pohybu otočením hodnoty rychlosti (`speedY`).

Nakonec se upraví jeho pozice tak, aby zůstal uvnitř herního pole a nezmizel mimo obrazovku.

```
if (newBallY <= 0 || newBallY >= 600 - 20) {
    ball.speedY = -ball.speedY;

    if (newBallY <= 0) newBallY = 0;
    else newBallY = 600 - 20;
}
```

5.3 Odraz míčku od pálek

Nejtěžší část projektu byla správně udělat odraz míčku od pálek tak, aby se nechoval **„nudně“** a aby záleželo na tom, kde míček pálku zasáhne. Po kolizi se proto vypočítá relativní místo zásahu na pálce **(od -1 do 1)**, z toho úhel odrazu v rozsahu **±60°** a zároveň se míček po každém odrazu mírně zrychlí. Díky tomu je hra postupně rychlejší a lépe se ovládá.

Technicky to funguje tak, že nejdříve určím **relativní pozici zásahu** (**L**) v rozmezí **-1** až **1** pomocí vzorce:

$$L = \frac{\left(y_{míčku} - y_{středu_{pálky}}\right)}{50}$$

(kde 50 je polovina výšky pálky).

Tuto hodnotu pak vynásobím maximálním úhlem 60° a pomocí funkcí sinus a cosinus přepočítám rychlost do os X a Y.

Původně byl systém odrazu jednodušší – pálka byla rozdělena pouze na tři části a míček se odrážel buď rovně, nahoru, nebo dolů. Tento způsob ale nepůsobil přirozeně a hra byla méně zajímavá. Proto jsem ho později vylepšil na plynulejší výpočet úhlu odrazu podle přesného místa zásahu. Nápad jsem měl, ale ještě jsem ho musel převést do kódu, s čímž mi pomohla umělá inteligence. **[6]**
Po detekci kolize s pravou pálkou se vypočítá relativní pozice zásahu hitPos v rozsahu -1 až 1, z níž se odvodí úhel odrazu (max. ±60°). Míček se zároveň o 5 % zrychlí:

```
    double hitPos = (ballCenter - paddleCenter) / 50.0;
    hitPos = Math.max(-1, Math.min(1, hitPos));

    double maxAngle = Math.toRadians(60);

    double angle = hitPos * maxAngle;
    speed *= 1.05;


    ball.speedX = -speed * Math.cos(angle);
    ball.speedY = speed * Math.sin(angle);
}
```

5.4 Skórování a reset míčku

Pokud míček přejde za levou nebo pravou hranu, přičte se bod soupeři a míček se resetuje doprostřed.

```
if (newBallX <= 0) {
    scoreRight++;
    resetBall();
    return;
}
```

```
if (newBallX >= 800 - 20) {
    scoreLeft++;
    resetBall();
    return;
}
```

Metoda `resetBall()`:

```
private void resetBall() {
    ball.x = 390;
    ball.y = 290;
    ball.speedX = 0;
    ball.speedY = 0;
    startDelay = 60;
}
```

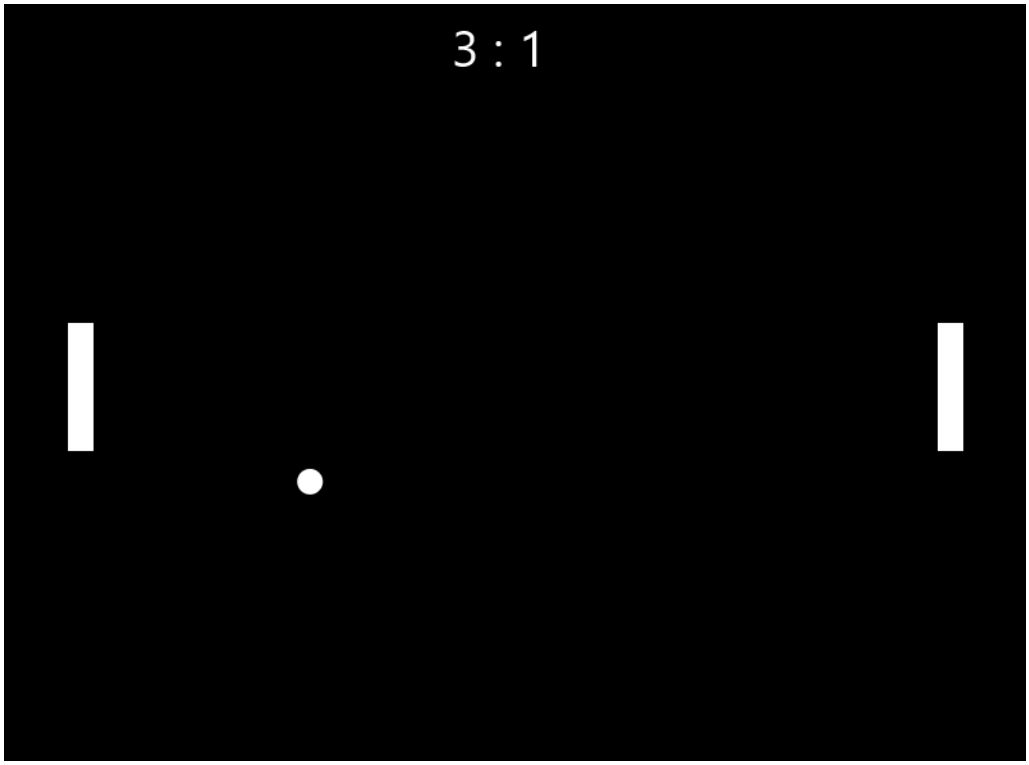
6 Uživatelské rozhraní a ovládání

6.1 Ovládání

Hra je navržena pro dva hráče na jedné klávesnici. Levý hráč ovládá svou pálku klávesami `W` (pohyb nahoru) a `S` (pohyb dolů), pravý hráč používá šipky `↑` a `↓`. Obě pálky lze ovládat současně, takže hra netrpí omezením střídání tahů. Po skončení hry lze novou partii spustit stisknutím klávesy `R`, aniž by bylo nutné aplikaci restartovat.

6.2 Zobrazení

Hra běží v okně o rozměrech 800 × 600 pixelů. Grafika je záměrně minimalistická – černé pozadí s bílými objekty odkazuje na vizuální styl původní arkádové hry Pong z roku 1972. Skóre obou hráčů je průběžně zobrazeno v horní části obrazovky uprostřed. Po dosažení pěti bodů jedním z hráčů se hra zastaví a uprostřed obrazovky se zobrazí jméno vítěze spolu s instrukcí pro restart. Ukázka herního okna během probíhající hry je zobrazena na obrázku 1.



Obrázek 1: Herní okno aplikace Pong během hry (skóre 3:1)

7 Požadavky a spuštění

7.1 Systémové požadavky

Pro spuštění aplikace je nutné mít na počítači nainstalovány následující součásti:

- **JDK 17 nebo novější** – doporučeno OpenJDK 21 nebo Oracle JDK 25 (použito v projektu)
- **JavaFX SDK 17+** – např. JavaFX 21 LTS (stažení na gluonhq.com/products/javafx)
- **IDE** – IntelliJ IDEA (použita verze 2025.3.2) nebo jiné Java IDE s podporou Maven/Gradle
- **Operační systém** – Windows, macOS nebo Linux

7.2 Instalace

Následující kroky popisují postup přípravy prostředí a načtení projektu:

3) Stáhněte nebo naklonujte zdrojový kód projektu (např. přes `git clone` nebo stažením ZIP archivu z repozitáře).
4) Nainstalujte JDK 25 (nebo jinou kompatibilní verzi) a nastavte proměnnou prostředí `JAVA_HOME` tak, aby ukazovala na adresář JDK.
5) Stáhněte JavaFX SDK a rozbalte ho do libovolného adresáře (např. `C:\javafx-sdk-21` na Windows nebo `/opt/javafx-sdk-21` na Linux/macOS).
6) Otevřete projekt v IntelliJ IDEA: *File* → *Open* a vyber složku projektu.
7) V IntelliJ nastav JavaFX knihovny: *File* → *Project Structure* → *Libraries* → klikni + a přidej cestu ke složce `lib` z JavaFX SDK.

7.3 Spuštění

Hru lze spustit dvěma způsoby:

Spuštění z IDE (IntelliJ IDEA)

8) Otevřete třídu `Launcher` v balíčku `com.example.pong`.
9) Pravým tlačítkem klikněte na třídu a zvolte *Run 'Launcher.main()'*.
10) Pokud se zobrazí chyba o chybějícím JavaFX modulu, přidejte do VM options (*Run* → *Edit Configurations*) následující příznak:

```
--module-path /cesta/k/javafx-sdk/lib --add-modules
javafx.controls,javafx.fxml
```

Spuštění z příkazové řádky

Pokud nemáte IDE, můžete aplikaci zkompilovat a spustit přímo příkazy:

```
javac --module-path $PATH_TO_FX --add-modules javafx.controls -d out
src/com/example/pong/*.java
```

```
java --module-path $PATH_TO_FX --add-modules javafx.controls -cp out
com.example.pong.Launcher
```

kde `$PATH_TO_FX` je cesta k adresáři `lib` v JavaFX SDK (např. `C:\javafx-sdk-21\lib`).

8 Závěr

V rámci projektu se podařilo vytvořit plně funkční hru Pong pro dva hráče v prostředí JavaFX. Implementoval jsem herní smyčku, vykreslování na plátno, ovládání klávesami a systém skórování s automatickým vyhlášením vítěze po dosažení stanoveného počtu bodů.

Zásadní problémy při řešení:

Nejnáročnější částí byl odraz míčku od pálek s výpočtem úhlu podle přesného místa zásahu. Zpočátku docházelo k chybám, kdy se míček při vysokých rychlostech občas „zasekl“ uvnitř pálky nebo jí proletěl. Tento problém jsem vyřešil přesným nastavením souřadnic ihned po kolizi a omezením maximálního úhlu odrazu na 60°, aby hra zůstala hratelná i při vyšším tempu.

Co se nepodařilo splnit:

Kvůli časové náročnosti ladění fyziky pohybu a plynulosti herní smyčky se nepodařilo implementovat komplexní menu a pokročilé grafické efekty. Projekt se tak v této fázi soustředí primárně na čistou hratelnost a technickou správnost pohybu.

Možný rozvoj projektu:

V budoucnu plánuji projekt dále rozšiřovat o hlavní menu (Start/Pause/Nastavení), které by zvýšilo uživatelský komfort. Velkým přínosem by byl AI protihráč pro hru jednoho hráče a přidání zvukových efektů. Také uvažuji o dalších herních režimech, jako je hra s více míčky nebo přidání pohyblivých překážek na hrací plochu.

9 Citace

[1] Wikipedia. *Pong* [online]. Wikimedia Foundation, 2024 [cit. 2026-04-13]. Dostupné z: https://en.wikipedia.org/wiki/Pong

[2] OpenJFX. *JavaFX – Open Source Client Application Platform* [online]. Gluon, 2024 [cit. 2026-04-13]. Dostupné z: https://openjfx.io/

[3] OpenJFX. *AnimationTimer (JavaFX 17 API Documentation)* [online]. OpenJFX, 2021 [cit. 2026-04-15]. Dostupné z: https://openjfx.io/javadoc/17/javafx.graphics/javafx/animation/AnimationTimer.html

[4] JetBrains. *IntelliJ IDEA – the Leading Java and Kotlin IDE* [online]. JetBrains, 2025 [cit. 2026-04-19]. Dostupné z: https://www.jetbrains.com/idea/

[5] OpenJDK. *JDK 25* [online]. Oracle, 2025 [cit. 2026-04-19]. Dostupné z: https://openjdk.org/projects/jdk/25/

[6] ANTHROPIC. *Claude* [online konverzační asistent]. Anthropic, 2025 [cit. 2025-04-23]. Výstup vygenerován na základě promptu: „Mám hru Pong. Chci, aby se míček odrážel od pálky podle toho, kde přesně ji zasáhne. Relativní pozici zásahu mám v rozsahu -1 až 1, maximální úhel odrazu chci 60°. Pomoz mi to převést do kódu pomocí funkcí sinus a cosinus tak, aby se zároveň míček po každém odrazu mírně zrychlil." Dostupné z: https://claude.ai